

Angular Micro Frontend Setup Guide

Module Federation + Angular Signals | Step-by-Step Commands

This guide walks you through setting up a complete Angular Micro Frontend (MFE) workspace with **Webpack Module Federation** (via @angular-architects/module-federation) and **Angular Signals** for cross-app state sharing.

Architecture Overview

App	Type	Port	Exposes
shell	Host	4200	—
dashboard	Remote	4201	./Component → DashboardViewComponent
product	Remote	4202	./Component → ProductListComponent
card	Remote	4203	./Component → InfoCardComponent

Step 1 — Create Workspace and Applications

Create a blank Angular workspace (no default app), then generate 4 standalone apps — one shell host and three remotes.

```
ng new multi-mfe-workspace --create-application=false --interactive=false cd
multi-mfe-workspace ng g application shell --routing=true --style=css --standalone=true
--interactive=false ng g application dashboard --routing=true --style=css --standalone=true
--interactive=false ng g application product --routing=true --style=css --standalone=true
--interactive=false ng g application card --routing=true --style=css --standalone=true
--interactive=false
```

Step 2 — Install and Initialise Module Federation

Use `ng add` to wire up module federation for each project. The shell is the **host**; the other three are **remotes**.

```
ng add @angular-architects/module-federation --project shell --port 4200 --type host
--interactive=false ng add @angular-architects/module-federation --project dashboard --port
4201 --type remote --interactive=false ng add @angular-architects/module-federation --project
product --port 4202 --type remote --interactive=false ng add
@angular-architects/module-federation --project card --port 4203 --type remote
--interactive=false
```

Note: This generates a `webpack.config.js` per project and patches `angular.json` to use the custom webpack builder.

Step 3 — Generate Remote Components

Scaffold a standalone component in each remote app. These are the components that will be exposed via Module Federation.

```
ng g component dashboard-view --project=dashboard --standalone=true --interactive=false ng g
component product-list --project=product --standalone=true --interactive=false ng g component
info-card --project=card --standalone=true --interactive=false
```

Step 4 — Create the Shared Global Signal Store

Create a shared signal store at the workspace root. It uses `window.__GLOBAL_STORE__` so the singleton is preserved across independently loaded MFE bundles.

File: `src/shared/global-store.ts`

```
mkdir -p src/shared # Create src/shared/global-store.ts with: import { signal, WritableSignal }
from '@angular/core'; export class GlobalStore { static selectedProduct:
WritableSignal<string> = signal('None'); } const globalRef = window as any; if
(!globalRef.__GLOBAL_STORE__) { globalRef.__GLOBAL_STORE__ = GlobalStore; } else {
GlobalStore.selectedProduct = globalRef.__GLOBAL_STORE__.selectedProduct; }
```

Note: The window trick prevents duplicate signal instances when Angular is loaded separately in each MFE bundle.

Step 5 — Add @shared/* Path Alias to tsconfig.json

Run this Node snippet once to add a `@shared/*` path alias so all projects can import from the shared store cleanly.

```
node -e 'const fs = require("fs"); const tsconfig =
JSON.parse(fs.readFileSync("tsconfig.json", "utf8")); if (!tsconfig.compilerOptions)
tsconfig.compilerOptions = {}; if (!tsconfig.compilerOptions.paths)
tsconfig.compilerOptions.paths = {}; tsconfig.compilerOptions.paths["@shared/*"] =
["src/shared/*"]; fs.writeFileSync("tsconfig.json", JSON.stringify(tsconfig, null, 2)); '
```

Step 6 — Configure Webpack Exposures for Each Remote

Patch each remote's `webpack.config.js` to expose its component at the `./Component` key.

Dashboard — `projects/dashboard/webpack.config.js`

```
exposes: { './Component':
'./projects/dashboard/src/app/dashboard-view/dashboard-view.component.ts', },
```

Product — `projects/product/webpack.config.js`

```
exposes: { './Component': './projects/product/src/app/product-list/product-list.component.ts',
},
```

Card — `projects/card/webpack.config.js`

```
exposes: { './Component': './projects/card/src/app/info-card/info-card.component.ts', },
```

Step 7 — Configure Shell Routes (Lazy-Load Remote Components)

File: `projects/shell/src/app/app.routes.ts`

```
import { Routes } from '@angular/router'; import { loadRemoteModule } from
'@angular-architects/module-federation'; export const routes: Routes = [ { path: 'dashboard',
loadComponent: () => loadRemoteModule({ type: 'module', remoteEntry:
'http://localhost:4201/remoteEntry.js', exposedModule: './Component' }).then(m =>
m.DashboardViewComponent) }, { path: 'products', loadComponent: () => loadRemoteModule({ type:
'module', remoteEntry: 'http://localhost:4202/remoteEntry.js', exposedModule: './Component'
}).then(m => m.ProductListComponent) }, { path: 'card', loadComponent: () => loadRemoteModule({
type: 'module', remoteEntry: 'http://localhost:4203/remoteEntry.js', exposedModule:
'./Component' }).then(m => m.InfoCardComponent) } ];
```

Step 8 — Create Shell Navigation UI

File: projects/shell/src/app/app.component.html

```
<nav style="display:flex; gap:20px; padding:15px; background:#222; color:white;"> <a
routerLink="/dashboard">Dashboard MFE</a> <a routerLink="/products">Products MFE</a> <a
routerLink="/card">Card MFE</a> </nav> <div style="padding:20px;"> <h2>Shell Frame
Platform</h2> <router-outlet></router-outlet> </div>
```

Step 9 — Implement Signal State — Sender (Product MFE)

The product app **writes** to the shared signal when a user clicks a button.

File: projects/product/src/app/product-list/product-list.component.ts

```
import { Component } from '@angular/core'; import { GlobalStore } from '@shared/global-store';
@Component({ selector: 'app-product-list', standalone: true, template: ` <button
(click)="select('M3 Macbook Pro')">Select MacBook</button> <button (click)="select('4K Gaming
Monitor')">Select Monitor</button> ` }) export class ProductListComponent { select(name:
string) { GlobalStore.selectedProduct.set(name); } }
```

Step 10 — Implement Signal State — Receiver (Dashboard MFE)

The dashboard app **reads** the shared signal reactively — no subscriptions or event buses needed.

File: projects/dashboard/src/app/dashboard-view/dashboard-view.component.ts

```
import { Component } from '@angular/core'; import { GlobalStore } from '@shared/global-store';
@Component({ selector: 'app-dashboard-view', standalone: true, template: ` <div> Live Data
Stream: <strong>{{ activeProduct() }}</strong> </div> ` }) export class DashboardViewComponent
{ activeProduct = GlobalStore.selectedProduct; }
```

Step 11 — Run the Full Environment

Open **4 separate terminal tabs** in the workspace root and run each command:

```
# Terminal 1 – Shell Host ng serve shell # Terminal 2 – Dashboard Remote ng serve dashboard #
Terminal 3 – Product Remote ng serve product # Terminal 4 – Card Remote ng serve card
```

Open `http://localhost:4200` in your browser. Navigate between routes to lazy-load each remote MFE.

Important Notes & Common Gotchas

- **Singleton signals:** The `window.__GLOBAL_STORE__` pattern ensures only one signal instance exists even when Angular is bundled separately in each remote.
- **Shared dependencies:** Make sure `@angular/core` is listed as a singleton shared dep in all `webpack.config.js` files (the library does this automatically via `ng add`).
- **tsconfig paths in remotes:** Each project's `tsconfig.app.json` inherits from the root `tsconfig.json`, so the `@shared/*` alias works across all apps.
- **Angular version:** This setup is compatible with Angular 14+. For Angular 17+ with esbuild, ensure `@angular-architects/module-federation v17+` is used.
- **Production builds:** Replace `localhost:420x` URLs in shell routes with your deployed CDN/server URLs for production.